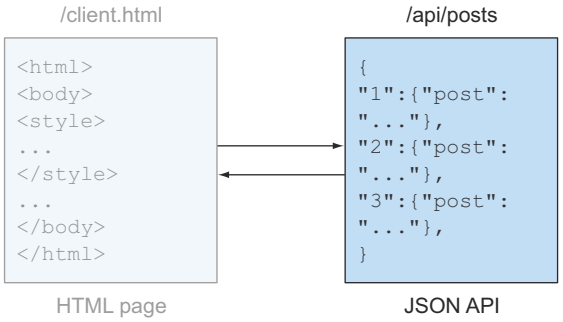


Figure 3.2 The location of the API endpoint within the sample app

blog posts (these are just made-up sample posts for the app; a real blog would load these posts from a database). It then creates a new Express web server and adds the `express.static()` middleware. The `express.static()` middleware configures the web server to read files from your computer and serve them through the web server (this will come into play in the next section, where you'll add a client.html file).

Next, the code sets up the actual API. The API lives at the endpoint /api/posts. Requests to /api/posts will return a JSON object containing the sample blog posts. Now that the code has configured how the server behaves, it starts the server on port 9999. Finally, the code prints out a friendly startup message to verify that everything is working.

Listing 3.1 Sample server code

```
var express = require('express');

var POSTS = {
  '1': {'post': 'This is the first blog post.'},
  '2': {'post': 'This is the second blog post.'},
  '3': {'post': 'This is the third blog post.'}
};

var SERVER_PORT = 9999;
var serverapp = express();
serverapp.use(express.static(__dirname));
serverapp.get('/api/posts', function(req, res) {
  res.json(POSTS);
});
serverapp.listen(SERVER_PORT, function() {
  console.log('Started server at http://127.0.0.1:' + SERVER_PORT);
});
```

Made-up blog posts for app

Creates a new Express-powered web server

Serves static files from the same directory as app.js

Adds API endpoint to retrieve sample posts

Starts server

Displays a startup message once server is started

NOTE If you are using Linux, you may have to invoke Node.js by typing `nodejs` rather than `node`.

You can run the server by opening a terminal window, navigating to the directory where your code lives, and typing `node app.js`. You should see the output in figure 3.3. You can stop the server by pressing Ctrl-C. Every time you make changes to the code in app.js, you'll need to stop and then start the server.